

Laboratorium Ilmu Rekayasa dan Komputasi

TEKNIK INFORMATIKA
STEI ITB

IF2224 - Formal Language and
Automata Theory

PASCAL-S Compiler

Milestone 3 - Semantic Analysis

Milestone 3

Semantic Analysis

Release : Selasa, 18 November 2025

Deadline : Minggu, 30 November 2025 pukul 23.59 WIB

Revisi : 2

Daftar Revisi

[25-11-2025] Revisi 1. Lampiran Reserved Words

[30-11-2025] Revisi 2. Perubahan nilai link yang sebelumnya salah pada contoh tabel tab

Daftar Isi

Daftar Revisi.....	2
Daftar Isi.....	4
I. Pendahuluan.....	5
II. Teori Singkat.....	10
III. Spesifikasi Tugas Besar.....	11
IV. Teknis Pengerjaan.....	21
V. Penilaian.....	23
VI. Pengumpulan.....	25
Referensi.....	26
Lampiran A. Link Penting.....	27
Lampiran B. Symbol Table.....	28

I. Pendahuluan



America's Got Techno 2025. New York City, New York

Beberapa hari telah berlalu setelah second phase dari Pascal-S compiler sudah selesai. Kendall Roy dan Gus Fring sangat bangga dengan effort kalian. Teknologi ini dipamerkan ke sebuah pameran tahunan US yang kebetulan lokasinya berada di New York. Waystar sebagai sebuah perusahaan berita yang berbasis di New York menyoroti acara ini. Namun, salah satu karyawan disitu tercengang melihat Kendall Roy menjadi bintang acara pada pameran tersebut bersama dengan Gus Fring. Mungkin karena ini adalah sebuah sukses yang sangat cepat didapat, Kendall Roy mengeluarkan kata-kata yang terkesan ingin menantang ketenaran Waystar dan Gojo. Dia seakan-akan belum bisa *move on* dari Greg dan Matsson yang mengeluarkan dia dari Waystar.



Ni orang ngeselin bet dah, kata Matsson (aproksimasi penulis)

Di kurun waktu yang cepat. Terkuak sebuah kasus bahwa Madrigal terafiliasi dengan sebuah operasi jual beli narkoba yang bernama metamfetamin (meth) dan Gus Fring diduga menggunakan *front* Los Pollos Hermanos untuk menyembunyikan operasinya, mencuci uang, dan mendukung distribusi metamfetamin tersebut. Matsson dengan koneksinya yang luas langsung memberikan serangan kuat ke pihak Madrigal dan Kendall Roy dan Gus Fring berada di dalam bahaya yang sangat besar. Mereka membutuhkan *backing* dari perusahaan yang bisa menutup operasi jual beli tersebut supaya orang-orang kembali berfokus kepada teknologi Pascal-S Compiler yang dibuat oleh kalian di Tecnicidad.



I know a guy who knows a guy

Mike Ehrmantraut konsultasi kembali ke teman lamanya Saul Goodman untuk mencari koneksi perusahaan yang kemungkinan bisa membantu Madrigal dari terjerat kasus jual beli narkoba. Saul Goodman berusaha mencari koneksi sebanyak-banyaknya dan sepertinya mendapatkan seseorang yang kemungkinan bisa membantu dan melindungi mereka.



Hughie dan pasangannya Starlight

Kedua orang tersebut meskipun tidak setuju dengan operasi jual beli narkoba ingin sekali mencari entitas besar yang bisa membantu mereka mengalahkan Vought International dan Homelander. Mereka terpaksa minta tolong ke Kendall Roy dan berdiskusi bagaimana cara mengalahkan Homelander. Namun, Kendall menginginkan supaya masalah dengan Waystar bisa diselesaikan terlebih dahulu. Hughie mengenalkan anda dan seluruh civitas Tecnicidad ke satu orang.



Stan Edgar

Bersama dengan Stan Edgar yang sangat intelektual, mereka berencana untuk menyerang Matsson dengan jurus yang sama yaitu mencari kesalahan. Namun, sepertinya Matsson ini seperti tidak punya noda dalam rekor trek nya

sehingga mencari kesalahan Matsson dan Gojo sepertinya mustahil. Namun tragedi tiba tiba muncul. Terdapat kabar bahwa adik dari Kendall Roy, Siobhan Roy (Shiv) diserang oleh seseorang yang memiliki janggut dan berbaju hitam yang sepertinya itu diklaim sebagai Billy Butcher, Waystar Gojo is under attack..... Matsson meskipun sepertinya tau siapa pelaku dibalik serangan tersebut tidak bisa mencari bukti yang konkret untuk menuduh Madrigal, Kendall, dan The Boys. Beberapa kali serangan ini terjadi dan ini membuat Matsson sangat marah dan kesal. Oleh karena itu, Kendall sempat diajak ke sebuah restoran untuk melakukan perdamaian. Namun, yang terjadi adalah Kendall diculik dan melihat adiknya yang satu lagi yaitu Roman Roy duduk dan diikat dan di sebelahnya berdirilah orang yang sangat tidak ingin dia hadapi.



Oh boy. What have you done....

Matsson telah bekerja sama dengan Vought International untuk menjatuhkan Madrigal dkk. Dan secara otomatis, dia berteman dengan Homelander. Homelander mengancam Kendall untuk mengeluarkan semua informasi terhadap operasi jual beli narkoba yang dijalankan oleh Gus Fring dan akan membunuh Roman dengan laser jika dia menolak untuk membeberkan rahasianya itu sekarang. Dia diberikan 3 hari untuk memberikan semua informasi konkrit terkait operasi jual beli narkoba tersebut. Sementara itu, anda sebagai karyawan di Tecnicidad akhirnya diminta oleh Gus untuk melakukan pekerjaan tersebut. Kali ini dia membutuhkan *third phase* karena dia berencana untuk bisa berkomunikasi dengan kompatiotnya yaitu Stan Edgar secara aman karena mereka berencana ingin menyelamatkan Kendall dan

Roman dari amarah Homelander. Oleh karena itu, por Los Tecnicidados. Marilah kita lanjut ke fase ketiga ini sebelum Homelander datang :)))))).

II. Teori Singkat

Sebelumnya, parser telah memastikan bahwa kode Pascal-S sudah benar secara sintaks berdasarkan aturan produksi grammar. Namun, perlu disadari bahwa sebuah grammar dapat menerima suatu program yang tidak masuk akal walaupun sintaksnya sudah benar. Akibatnya, perlu dilakukan pemeriksaan lebih lanjut apakah struktur program tersebut juga benar secara makna (semantic).

Maka dari itu, pada tahap berikutnya akan dilakukan semantic analysis untuk memeriksa konsistensi tipe data, deklarasi variabel dan fungsi, lingkup (scope) program, serta control flow yang sesuai dengan aturan bahasa.

Semantic analysis akan melakukan anotasi terhadap parse tree yang telah dihasilkan. Anotasi berupa (dan tak terbatas kepada) penambahan informasi seperti tipe data dan referensi simbol (identifier) ke symbol table.

Beberapa hal yang perlu dipahami:

1. Attributed Grammar

Grammar yang dilengkapi dengan atribut dan aturan semantik.

2. Symbol Table

Struktur data yang minimal menyimpan informasi nama identifier, tipe datanya dan scopenya. **Symbol table menggunakan struktur data stack.**

3. Visitor / Visit Functions

Untuk setiap jenis node akan memiliki fungsi visit_<node> yang akan:

- 1) Memanggil visit pada child-nya
- 2) Mengakses atau memperbarui symbol table
- 3) Menghitung atribut/tipe semantik
- 4) Menambahkan anotasi pada node

III. Spesifikasi Tugas Besar

Pada milestone ini, Anda diminta untuk membuat **tahapan ketiga dari compiler**, yaitu **semantic analysis**. Tahapan ini berfungsi melakukan analisis makna (semantic analysis) dengan menggunakan **Attributed Grammar**.

Parse tree yang sudah dihasilkan akan ditelusuri secara top-down menggunakan fungsi visit (semantic visitor) pada setiap node-nya yang dibuat berdasarkan grammar yang sebelumnya didefinisikan. Selama proses ini, symbol table digunakan untuk menyimpan dan mengambil informasi deklarasi simbol/identifier.

Masukan	Parse Tree
Keluaran	Decorated AST, Symbol Table
Algoritma	L-Attributed Grammar

Pada tahap ini akan dilakukan hal sebagai berikut:

1. **Membentuk Abstract Syntax Tree**, termasuk:
 - a) Memanfaatkan Syntax-Directed Translation Scheme.
 - b) Identifikasi production rule dan semantic action yang terkait.
 - c) Menjalankan semantic action pada node parser tree, akan terbentuk node baru untuk AST.
2. **Type & Scope Checking**, meliputi:
 - a) Mengunjungi setiap node AST secara Depth First.
 - b) Push scope baru pada stack symbol table apabila memasuki block kode baru.
 - c) Memasukan variabel/fungsi pada symbol table untuk setiap deklarasi.
 - d) Setiap bertemu dengan identifier, lookup pada symbol table.
 - d) Kalkulasi type data dari node yang dikunjungi
 - e) Validasi apakah setiap type sesuai semantik, lalu "dekorasi" AST.
 - f) Hasil dari tahap ini adalah program yang sudah tervalidasi secara semantik serta **Decorated AST**, dimana setiap node minimal memiliki informasi:
 - i) Tipe ekspresi (lihat pada ["Semantic Analysis \(ULiège\)"](#) bagaimana tipe ekspresi diperoleh)

- ii) Referensi ke symbol table
- iii) Informasi scope

Sebelum memulai proses, inisiasi symbol table terlebih dahulu. Pada Pascal-S terdapat 3 jenis symbol table:

1. **tab**: digunakan untuk menyimpan informasi mengenai identifier termasuk konstanta, variabel, prosedur, atau fungsi.
2. **btab**: digunakan untuk menyimpan informasi blok prosedur dan definisi tipe rekord.
3. **atab**: digunakan untuk menyimpan informasi array seperti tipe index, batasan index, tipe elemen, dan ukuran array.

Contoh symbol table tersedia pada [Lampiran B. Symbol Table](#)

Berikut merupakan daftar atribut dari symbol table

tab	
atribut	deskripsi
identifiers	Nama identifier (misalnya nama variabel, konstanta, tipe, prosedur, fungsi). Indeks dimulai dari 29 karena ada reserved words.
link	Pointer/indeks ke identifier sebelumnya dalam scope yang sama. Digunakan untuk manajemen scope (linked list per blok).
obj	Kelas objek yang dienumerasi: konstanta, variabel, tipe, prosedur, fungsi, dll.
type	Tipe dasar dari identifier, misalnya: integer, boolean, char, real, array, record, dll. Biasanya berupa kode numerik.
ref	Pointer/indeks ke tabel lain jika tipe adalah komposit (array/record). Mengarah ke atab (array table) atau btab (record/procedure block).
nrm	Menandai apakah identifier adalah variabel normal (=1) atau parameter by-reference (var parameter) (=0).
lev	Tingkat <i>lexical level</i> tempat identifier dideklarasikan (0 = global, 1 = dalam prosedur, 2 = dalam prosedur di dalam prosedur, dst).
adr	Makna tergantung jenis objek: offset variabel di stack frame, nilai

	konstanta, offset field record, alamat prosedur, atau ukuran/penanda lain.
--	--

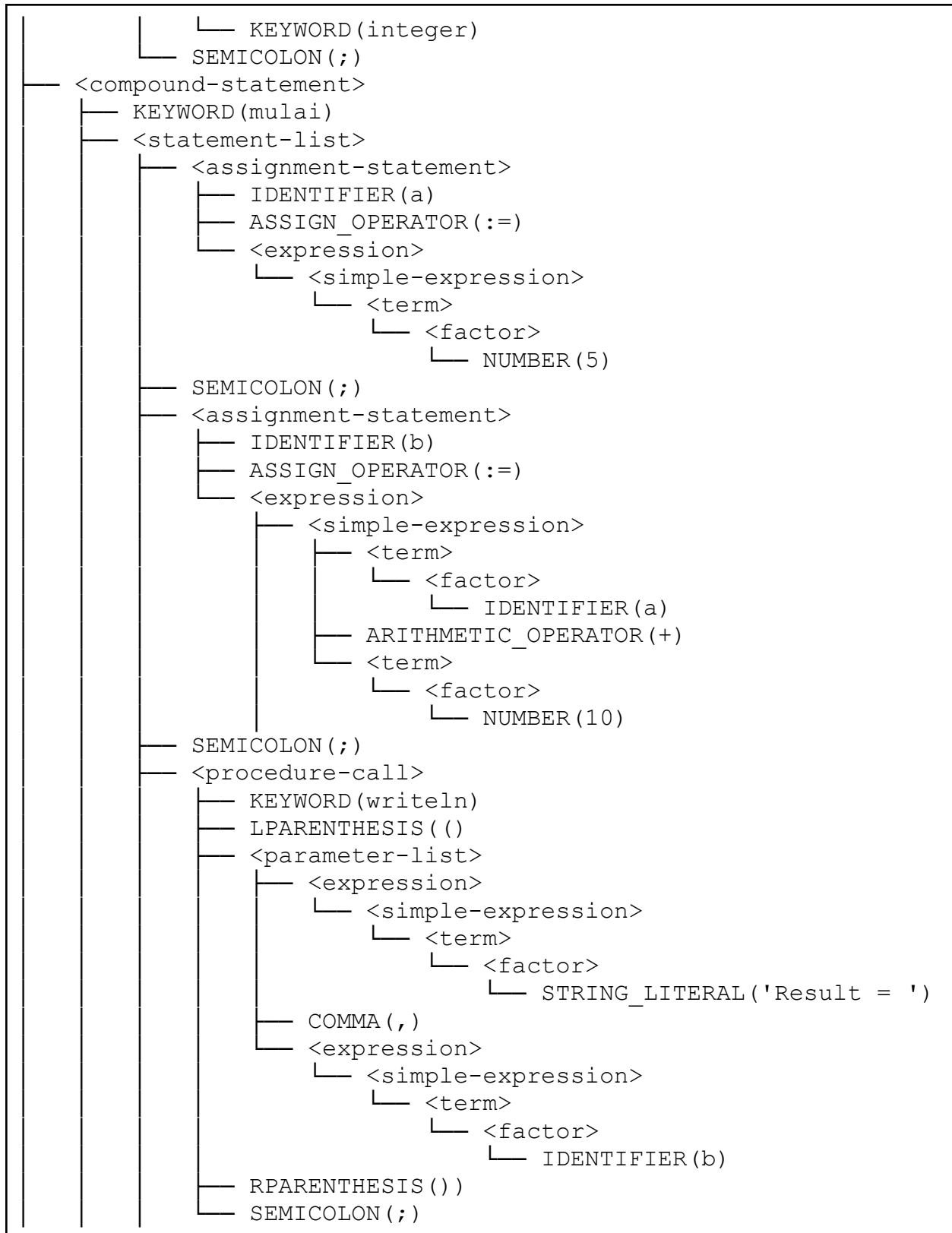
atab	
atribut	deskripsi
arrays	Indeks entri array
xtyp	Tipe indeks array (misalnya integer). Berupa kode tipe dari tabel tab.
etyp	Tipe elemen array (misalnya integer). Berupa kode tipe dari tabel tab.
eref	Pointer/indeks ke detail tipe elemen jika elemen adalah tipe komposit (misalnya array dalam array, atau record). Mengarah ke atab atau btab.
low	Batas bawah indeks array (misalnya 1 pada array[1..10] atau 0 pada array[0..15]).
high	Batas atas indeks array.
elsz	Ukuran satu elemen array (dalam byte/unit memori).
size	Total ukuran array

btab	
atribut	deskripsi
blocks	Indeks entri block (setiap block mewakili prosedur, fungsi, atau record type definition).
last	Pointer/indeks ke identifer terakhir yang dideklarasikan di dalam block tersebut (menghubungkan field record, parameter, atau variabel lokal).
lpar	Pointer/indeks ke parameter terakhir dari prosedur/fungsi pada block tersebut. Jika block adalah record, nilainya 0.
psze	Total ukuran parameter block (dalam byte/unit memori).

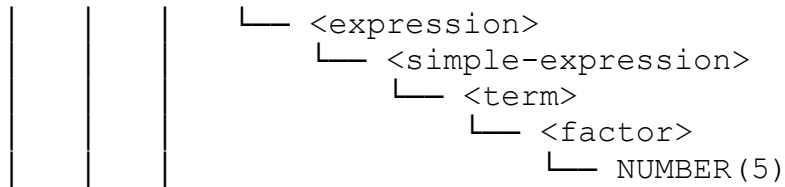
vsze	Total ukuran variabel lokal block (dalam byte/unit memori)
------	--

Dengan memiliki informasi pada symbol table, dapat dipastikan konsistensi tipe data serta memastikan variabel tidak bisa dioperasikan jika belum diinisialisasi dengan nilai.

Input Command (contoh yang digunakan menggunakan Python) Format: python [Compiler] [Kode Pascal]
python compiler.py program.pas
Isi program.pas
<pre> program Hello; variabel a, b: integer; mulai a := 5; b := a + 10; writeln('Result = ', b); selesai.</pre>
Masukan (Input) untuk fase semantic analysis
Parse tree yang dihasilkan oleh parser
Isi Parse tree yang sebelumnya dihasilkan oleh parser
<pre> <program> ├── <program-header> │ ├── KEYWORD(program) │ ├── IDENTIFIER(Hello) │ └── SEMICOLON(;) ├── <declaration-part> │ └── <var-declaration> │ ├── KEYWORD(variabel) │ ├── <identifier-list> │ │ ├── IDENTIFIER(a) │ │ ├── COMMA(,) │ │ └── IDENTIFIER(b) │ ├── COLON(:) │ └── <type></pre>



└─ KEYWORD(selesai) └─ DOT(.)	
Semantic Rule yang digunakan	
Production Rule	Semantic Rule
<code><assignment-statement> → ID := <expr></code>	<code>assignment_statement.node = new AssignNode(new VarNode(ID.lexeme), expr.node)</code>
<code><expression> → <simple-expr> + <term></code>	<code>expression.node = new BinOpNode('+', simple_expr.node, term.node)</code>
<code><expression> → <simple-expr></code>	<code>expression.node = simple_expr.node</code>
<code><factor> → NUMBER</code>	<code>factor.node = new NumberNode(NUMBER.value)</code>
<code><factor> → ID</code>	<code>factor.node = new VarNode(ID.lexeme)</code>
<code><term> → <factor></code>	<code>term.node = factor.node</code>
<code><factor> → STRING_LITERAL</code>	<code>factor.node = new StringNode(STRING_LITERAL.value)</code>
<code><procedure-call> → ID (<params>)</code>	<code>procedure_call.node = new ProcCallNode(ID.lexeme, params.nodes)</code>
Proses Pembangunan AST	
└─ <assignment-statement> └─ IDENTIFIER(a) └─ ASSIGN_OPERATOR(:=)	



Misalkan Syntax Analyzer sedang memeriksa sub-tree diatas dan masuk sampai kedalam node "NUMBER(5)"

Production Rule yang digunakan adalah $\langle \text{factor} \rangle \rightarrow \text{ID}$ dengan semantic rule `factor.node = new VarNode(ID.lexeme)`

Maka, aksi yang dilakukan adalah
`factor.node = new NumberNode(5)`
 dan terbentuk node baru pada AST yaitu
`NumberNode(5)`

Kemudian naik pada node "<term>"

Production rule yang digunakan $\langle \text{term} \rangle \rightarrow \langle \text{factor} \rangle$

Maka digunakan `term.node = factor.node` yang berakibat
`term.node = NumberNode(5)`

Atribut dinaikkan dalam tree (synthetized) sampai diperoleh bahwa `expression.node = NumberNode(5)`

Sekarang Syntax Analyzer naik sampai node

$\langle \text{assignment-statement} \rangle$ dimana production rulanya

$\langle \text{assignment-statement} \rangle \rightarrow \text{ID} := \langle \text{expr} \rangle$

Semantic rulanya adalah

`assignment_statement.node = new AssignNode(new
 VarNode(ID.lexeme), expr.node)`

Karena node ini punya leaf IDENTIFIER(a) dan attribut
 synthesized dari $\langle \text{expression} \rangle$ maka terjadi

`assignment_statement.node = new AssignNode(new VarNode('a'),
 NumberNode(5))`

Dan terbentuk node AST baru yaitu

`AssignNode(target: VarNode('a'), value: NumberNode(5))`

AST yang dihasilkan

```

ProgramNode(name: 'Hello')
|
+-- Declarations
|   |
|   +-- VarDecl(name: 'a', type: 'integer')
|   \-- VarDecl(name: 'b', type: 'integer')
|

```

```

\-- Block
|
|-- Assign(target: Var('a'), value: Num(5))
|
|-- Assign(target: Var('b'),
|           value: BinOp(op: '+',
|                       left: Var('a'),
|                       right: Num(10)))
|
\-- ProcedureCall(name: 'writeln',
                  args: [String('Result = '), Var('b')])

```

Proses Type & Scope Checking

Traversal dimulai dari root dengan fungsi visit(ProgramNode). Sebelum memulai, inisialisasi ketiga tabel simbol sesuai spesifikasi Pascal-S:

```

tab = []    # identifier table (mulai dari indeks 29 untuk
reserved words)
btab = [    # block table - indeks 0 adalah program (global
block)
    { "last":0, "lpar":0, "psize":0, "vsze":0 }    # btab[0] =
global block
]
atab = []   # array table (kosong awal)

display = [0]    # display[level] = indeks btab dari block pada
level tersebut
level    = 0     # lexical level (0 = global)

```

Contoh proses untuk program Hello yang sama:

1. visit(ProgramNode name='Hello')
 - Buat block baru untuk program (global block) → sudah ada di btab[0]
 - Masukkan nama program ke tab:


```

tab.append({
    "id": "Hello", "obj": "program", "type": 0, "ref": 0,
    "nrm": 1, "lev": 0, "adr": 0, "link": 0
})

```
 - btab[0]["last"] = indeks terakhir di tab yang termasuk block ini
2. visit(Declarations)

```

    • visit(VarDecl 'a', type='integer')
      • Cari tipe integer di tab (reserved, misal indeks 1 = integer)
      • Masukkan ke tab:
        tab.append({
          "id": "a", "obj": "variable", "type": 1, "ref": 0,
          "nrm": 1, "lev": 0, "adr": 0, "link":
previous_var_in_block
        })
      • Update btab[0]["last"] = indeks baru ini
      • adr akan dihitung nanti saat code generation (ukuran tiap variabel = 1 untuk integer)

    • visit(VarDecl 'b', type='integer')
      → sama seperti a, link menunjuk ke entri a (linked-list dalam block)

3. visit(Block) - masuk ke compound statement (main block)
  • level += 1
  • Buat block baru di btab:
    btab.append({
      "last": 0, "lpar": 0, "psze": 0, "vsze": 0
    })
  • display.append(len(btab)-1) # simpan indeks block level 1

4. visit(Assign target='a', value=5)
  • visit(Var 'a')
    • Lookup identifier 'a':
      mulai dari block saat ini (display[level]=1), ikuti link di btab
      → tidak ada → turun ke level 0 → ketemu di tab[indeks_a]
    • Anotasi node: type = integer, tab_index = indeks_a, lev = 0
  • visit(Number 5) → type = integer
  • Cek kompatibilitas tipe → ok
  • Tandai variabel a sebagai initialized (bisa ditambah field atau flag terpisah)

5. visit(Assign target='b', value=BinOp('+', Var('a'), 10))
  • Sama seperti di atas untuk 'b'
  • BinOp '+': kedua operand integer → result type integer
  • Assignment: target type = value type → ok

```

```
6. visit(ProcedureCall 'writeln')
  • Lookup 'writeln' → predefined procedure (sudah ada di tab indeks kecil)
  • Cek jumlah dan tipe parameter (string + integer) → sesuai
```

Keluaran (output)

tab (hanya sebagian yang relevan):

idx	id	obj	type	ref	nrm	lev	adr	link

...	(reserved words 0-28)							
29	Hello	program	0	0	1	0	0	0
30	a	variable	1	0	1	0	0	0
31	b	variable	1	0	1	0	0	30
32	writeln	procedure	... (predefined)					

btab:

idx	last	lpar	psze	vsze	

0	31	0	0	2	← global block (program), vsze=2 (a dan b)
1	0	0	0	0	← main block (kosong variabel lokal)

atab: (kosong karena tidak ada array)

Decorated AST (contoh anotasi minimal):

```
ProgramNode(name: 'Hello')
├── Declarations
│   ├── VarDecl('a') → tab_index:30, type:integer, lev:0
│   └── VarDecl('b') → tab_index:31, type:integer, lev:0
├── Block → block_index:1, lev:1
│   ├── Assign('a' := 5) → type:void
│   │   ├── target 'a' → tab_index:30, type:integer
│   │   └── value 5 → type:integer
│   ├── Assign('b' := a+10) → type:void
│   │   ├── target 'b' → tab_index:31, type:integer
│   │   └── BinOp '+' → type:integer
│   │       ├── 'a' → tab_index:30, type:integer
│   │       └── 10 → type:integer
│   └── writeln(...) → predefined, tab_index:32
```

IV. Teknis Pengerjaan

- Bahasa pemrograman yang digunakan **disamakan** dengan pengerjaan milestone sebelumnya.
- Pengerjaan dilakukan pada **repositori yang sama** dengan milestone sebelumnya.
- Anda **WAJIB** membuat **laporan** dengan format **Laporan-X-[KODE].pdf**, dengan **X** adalah **nomor Milestone** dan **Kode** adalah **Kode Kelompok**.
- **Laporan setidaknya berisi:**
 - Cover dengan foto kelompok menggantikan logo ITB.
 - Landasan Teori
 - Jelaskan teori-teori atau konsep yang berhubungan atau digunakan pada Milestone 3.
 - Perancangan & Implementasi
 - Implementasi yang ditambahkan dan berhubungan dengan Milestone 3.
 - Pengujian
 - Setidaknya 5 pengujian kasus unik.
 - Setiap pengujian harus menunjukkan hasil input/output program.
 - Bukti berupa screenshot input/output program.
 - Kesimpulan dan Saran
 - Lampiran berupa:
 - Link Release Repository Github.
 - Pembagian Tugas (menunjukkan persentase kontribusi).
 - Referensi yang digunakan selama pengerjaan Milestone 3
- Jika ada pertanyaan lebih lanjut, silahkan ditanyakan pada [QNA](#).
- Link Pembagian Kelompok: [Sheets Pembagian Kelompok](#)

V. Penilaian

Program (70 poin)			
No	Komponen	Poin	Syarat Poin Maksimal
1.	Implementasi visit functions	12	Semua visit function lengkap & sesuai grammar.
2.	Implementasi symbol table	10	Symbol table menyimpan atribut minimal & berfungsi dengan benar saat proses semantic analysis.
3.	Annotated AST	10	Parse tree sudah teranotasi sesuai aturan AST dan tipe datanya konsisten.
4.	Struktur/modularitas program	8	Kode ditulis dengan struktur modular menggunakan fungsi atau kelas yang terpisah berdasarkan tanggung jawab. Alur eksekusi program mudah diikuti dan tidak redundan
5.	Error handling	5	Program dapat menangani error dengan baik (pesan informatif dan error tidak menyebabkan crash)
6.	Penerimaan input	3	Program dapat membaca file input hasil tokenisasi pada milestone 1 (.txt)
7.	Penghasilan output	2	Program mengeluarkan output Parse Tree yang terformat dengan jelas ke terminal
8.	Dokumentasi/komentar	5	Kode diberi komentar yang informatif dan tidak berlebihan
9.	Pengujian	15	Program berhasil lolos seluruh kasus uji

Laporan (30 poin)			
No	Komponen	Poin	Syarat Poin Maksimal
1.	Penjelasan konsep	20	Penjelasan mengenai konsep semantic analysis.
2.	Perancangan program	5	Penjelasan mengenai rancangan program yang mencakup teknologi yang digunakan, struktur program, fungsi/kelas utama, dan alur kerja program. Terdapat pula justifikasi atas pilihan implementasi
3.	Hasil implementasi	2	Dokumentasi mengenai hasil implementasi, dapat berupa tangkapan layar dengan penjelasan
4.	Test case	3	Dokumentasi mengenai pengujian mandiri (minimal 5 dan mencakup berbagai kasus)

VI. Pengumpulan

- **Deliverables:**
 - Laporan
 - Release Repository
- Pada setiap Milestone, Anda **WAJIB** mengumpulkan **Link menuju Release pada Repository Github.**
 - Gunakan format tag berupa **v0.X.Y** dengan **X** adalah **nomor milestone** dan **Y** adalah **nomor pengumpulan**.
 - Sebagai contoh, untuk milestone 1, release pertama, dan pengumpulan pertama adalah **v0.1.1**. Untuk pengumpulan selanjutnya karena revisi, gunakan tag **v0.1.2** dan seterusnya sesuai nomor pengumpulan. Untuk milestone selanjutnya, pengumpulan dimulai kembali dari satu yaitu **v0.2.1, v0.2.2**, dan seterusnya.
- **Laporan** dikumpulkan dengan cara **meletakkan** hasil **PDF** dalam **folder doc** dengan format **Laporan-X-[KODE].pdf**, dimana **X** adalah **nomor milestone**. Pastikan isi laporan sudah sesuai dengan [teknis pengerjaan](#).
- Link Pengumpulan: [Pengumpulan Milestone](#)
- Deadline milestone: **Minggu, 30 November 2025 pukul 23.59 WIB**

Referensi

"PASCAL-S: A Subset and its implementation"

<http://pascal.hansotten.com/uploads/pascals/PASCAL-S%20A%20subset%20and%20its%20Implementation%20012.pdf>

"Let's Build A Simple Interpreter. Part 7: Abstract Syntax Trees"

<https://ruslanspivak.com/lbasi-part7>

"Contoh Gambar Parse Trees"

https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png

"Semantic Analysis (ULiège)"

<https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>

"Pascal for small machines"

<http://pascal.hansotten.com/niklaus-wirth/pascal-s/>

Lampiran A. Link Penting

Laman Dokumen Terpusat: [☰ Dokumen Terpusat - Tubes IF2224 TBFO](#)

Laman Pembagian Kelompok: [📁 Pembagian Kelompok - Tubes IF2224 TBFO](#)

Laman QnA Tugas Besar: [📁 QNA - Tubes IF2224 TBFO](#)

Form Pengumpulan: [Form Pengumpulan - Tubes IF2224 TBFO](#)

Lampiran B. Symbol Table

Berikut merupakan tangkapan layar dari buku "PASCAL-S: A Subset and its implementation" Halaman 15-17

sample program Pascal-S

```

0  program test0(output);
0  const ten = 10; plus = '+';
0  type row = array [1..ten] of real;
0      complex = record re,im: real and ;
0  var i,j: integer;
0      p: boolean;
0      z: complex;
0      matrix: array [-3..+3] of row;
0      pattern: array [1..5, 1..5] of char;
0

```

-16-

```

0  procedure dummy(var i: integer; var z: complex);
0      var u,v: row;
0      h1,h2: record c: complex; r: row
0      end ;
0
0      function null(x,y: real; z: complex): boolean;
0      var a: array ['a'..'z'] of complex;
0      u: char;
0      begin while x < y do x := x+1.0;
10      null := x=y
12      end (*null*) ;
15
15      begin p := null(h1.c,re, h2.c.im, z)
25      end (*dummy*) ;
27
27      begin i := 85; j := 51;
34      repeat
34          if i > j then i := i-j else j := j-i
46      until i = j;
53      writeln(i)
55      end .

```

tab yang dihasilkan

identifiers	link	obj	typ	ref	nrm	lev	adr
29 ten	0	0	1	0	0	1	10
30 plus	29	0	4	0	0	1	37
31 row	30	2	5	1	0	1	10
32 complex	31	2	6	3	0	1	2
33 re	0	1	2	0	1	2	0
34 im	33	1	2	0	1	2	1
35 i	32	1	1	0	1	1	5
36 j	35	1	1	0	1	1	6
37 p	36	1	3	0	1	1	7
38 z	37	1	6	3	1	1	8
39 matrix	38	1	5	2	1	1	10
40 pattern	39	1	5	3	1	1	80
41 dummy	40	3	0	4	1	1	16
42 i	0	1	1	0	0	2	5
43 z	42	1	6	3	0	2	6
44 u	43	1	5	1	1	2	7
45 v	44	1	5	1	1	2	17
46 h1	45	1	6	5	1	2	27
47 h2	46	1	6	5	1	2	39
48 c	0	1	6	3	1	3	0
49 r	48	1	5	1	1	3	2
50 null	47	4	3	6	1	2	0
51 x	0	1	2	0	1	3	5
52 y	51	1	2	0	1	3	6
53 z	52	1	6	3	1	3	7
54 a	53	1	5	5	1	3	9
55 u	54	1	4	0	1	3	61

btab yang dihasilkan

blocks	last	lpar	psze	vsze
1	28	1	0	0
-17-				
2	41	28	5	105
3	34	0	0	2
4	50	43	7	51
5	49	0	0	12
6	55	53	9	62

atab yang dihasilkan

arrays	xtyp	etyp	eref	low	high	elsz	size
1	1	2	0	1	10	1	10
2	1	5	1	-3	3	10	70
3	1	5	4	1	5	5	25
4	1	4	0	1	5	1	5
5	4	6	3	1	26	2	52

Lampiran C. Reserved Words

1. AND
2. ARRAY
3. BEGIN
4. CASE
5. CONST
6. DIV
7. DOWNTO
8. DO
9. ELSE
10. END
11. FOR
12. FUNCTION
13. IF
14. MOD
15. NOT
16. OF
17. OR
18. PROCEDURE
19. PROGRAM
20. RECORD
21. REPEAT
22. STRING
23. THEN
24. TO
25. TYPE
26. UNTIL
27. VAR
28. WHILE
29. PACKED

Sumber: <http://pascal.hansotten.com/niklaus-wirth/pascal-s/>